

# Final Java Capstone Submission & Interview Readiness Review

This is the **final checkpoint** of your Java internship roadmap. The goal is not to add more advanced technology, but to prove that you can take the Java concepts you've learned and build a **clean, testable, maintainable backend application**.

---

## 1. Final Capstone Structure

For a Maven Java project, a good structure is:

```
library-management/
├── pom.xml
├── README.md
├── src/
│   ├── main/
│   │   └── java/
│   │       └── com/example/library/
│   │           ├── app/
│   │           │   └── LibraryApplication.java
│   │           ├── model/
│   │           │   ├── Book.java
│   │           │   └── User.java
│   │           ├── dto/
│   │           │   ├── BookRequest.java
│   │           │   └── BookResponse.java
│   │           ├── service/
│   │           │   └── BookService.java
│   │           ├── repository/
│   │           │   └── BookRepository.java
│   │           └── exception/
│   │               ├── BookNotFoundException.java
│   │               └── GlobalExceptionHandler.java
```

```
├── util/
│   └── BookMapper.java
└── test/
    ├── java/
    │   ├── com/example/library/
    │   │   └── service/
    │   │       └── BookServiceTest.java
```

You don't have to create every package if your project doesn't need it.

The important thing is **clear responsibility**.

---

## 2. What Each Package Does

### model

Contains your main domain objects.

Example:

```
public class Book {

    private Long id;
    private String title;
    private String author;
    private boolean available;

    public Book(Long id, String title, String author, boolean available) {
        this.id = id;
        this.title = title;
        this.author = author;
        this.available = available;
    }

    public Long getId() {
        return id;
    }

    public String getTitle() {
        return title;
    }
}
```

```
}

    public String getAuthor() {
        return author;
    }

    public boolean isAvailable() {
        return available;
    }

    public void setAvailable(boolean available) {
        this.available = available;
    }
}
```

---

## 3. dto Package

DTO means:

### Data Transfer Object

It represents data moving into or out of your application.

Example request:

```
public class BookRequest {

    private String title;
    private String author;

    public BookRequest(String title, String author) {
        this.title = title;
        this.author = author;
    }

    public String getTitle() {
        return title;
    }

    public String getAuthor() {
        return author;
    }
}
```

```
}  
}
```

Response:

```
public class BookResponse {  
  
    private Long id;  
    private String title;  
    private String author;  
  
    public BookResponse(Long id, String title, String author) {  
        this.id = id;  
        this.title = title;  
        this.author = author;  
    }  
  
    public Long getId() {  
        return id;  
    }  
  
    public String getTitle() {  
        return title;  
    }  
  
    public String getAuthor() {  
        return author;  
    }  
}
```

The idea is:

```
Request DTO  
  ↓  
Service  
  ↓  
Model  
  ↓  
Response DTO
```

---

## 4. Repository Layer

If you're using collection persistence:

```
import java.util.ArrayList;
import java.util.List;

public class BookRepository {

    private final List books = new ArrayList<>();

    public Book save(Book book) {
        books.add(book);
        return book;
    }

    public List findAll() {
        return new ArrayList<>(books);
    }

    public Book findById(Long id) {

        return books.stream()
            .filter(book -> book.getId().equals(id))
            .findFirst()
            .orElse(null);
    }

    public void delete(Book book) {
        books.remove(book);
    }
}
```

The important point is that the service shouldn't care whether the data is stored in:

ArrayList

or:

MySQL

The repository hides that implementation.

---

## 5. JDBC Difficulty

If you selected JDBC persistence instead of collections, your structure might be:

```
BookService
  ↓
BookRepository
  ↓
JDBC
  ↓
MySQL
```

For example:

```
public class BookRepository {

    public Book findById(Long id) {

        String sql = "SELECT id, title, author FROM books WHERE id = ?";

        // JDBC connection
        // PreparedStatement
        // ResultSet
        // Mapping
        // Return Book

        return null;
    }
}
```

You should use:

```
PreparedStatement
```

rather than concatenating user input into SQL.

---

## 6. Service Layer

The service contains business logic.

Example:

```
import java.util.List;

public class BookService {

    private final BookRepository bookRepository;

    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public Book addBook(String title, String author) {

        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Title is required");
        }

        if (author == null || author.isBlank()) {
            throw new IllegalArgumentException("Author is required");
        }

        Book book = new Book(
            System.currentTimeMillis(),
            title,
            author,
            true
        );

        return bookRepository.save(book);
    }

    public List getAllBooks() {
        return bookRepository.findAll();
    }

    public Book getBook(Long id) {

        Book book = bookRepository.findById(id);

        if (book == null) {
```

```
        throw new BookNotFoundException(id);
    }

    return book;
}
}
```

Notice:

```
Controller/HTTP
  ↓
BookService
  ↓
BookRepository
```

The service doesn't deal with HTTP.

---

## 7. Custom Exception

```
public class BookNotFoundException extends RuntimeException {

    public BookNotFoundException(Long id) {
        super("Book not found: " + id);
    }
}
```

This is better than scattering:

```
throw new RuntimeException(...)
```

everywhere.

Your application can have meaningful exceptions for meaningful failures.

---

## 8. Application Layer

Your app package contains the entry point.

For a simple Java application:

```
public class LibraryApplication {  
  
    public static void main(String[] args) {  
  
        BookRepository repository = new BookRepository();  
  
        BookService service =  
            new BookService(repository);  
  
        service.addBook(  
            "Clean Code",  
            "Robert C. Martin"  
        );  
  
        System.out.println(service.getAllBooks());  
    }  
}
```

This manually wires the dependencies:

```
BookRepository  
  ↓  
BookService
```

Later, a framework such as Spring can automate this wiring.

---

## 9. Testing Service Logic

Your capstone should contain meaningful JUnit tests.

For example:

```
import org.junit.jupiter.api.Test;  
  
import static org.junit.jupiter.api.Assertions.*;
```

```

class BookServiceTest {

    @Test
    void shouldAddBook() {

        BookRepository repository = new BookRepository();
        BookService service = new BookService(repository);

        Book book = service.addBook(
            "Clean Code",
            "Robert C. Martin"
        );

        assertNotNull(book);
        assertEquals("Clean Code", book.getTitle());
    }

    @Test
    void shouldRejectBlankTitle() {

        BookRepository repository = new BookRepository();
        BookService service = new BookService(repository);

        assertThrows(
            IllegalArgumentException.class,
            () -> service.addBook("", "Author")
        );
    }

    @Test
    void shouldThrowWhenBookDoesNotExist() {

        BookRepository repository = new BookRepository();
        BookService service = new BookService(repository);

        assertThrows(
            BookNotFoundException.class,
            () -> service.getBook(999L)
        );
    }
}

```

The important thing isn't simply:

"I have tests."

It's:

"My tests verify meaningful behavior."

---

## 10. Mockito Test

If you're using Mockito, you can isolate the service from the repository.

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

class BookServiceTest {

    @Test
    void shouldGetBookFromRepository() {

        BookRepository repository =
            mock(BookRepository.class);

        Book book =
            new Book(1L, "Clean Code", "Robert C. Martin", true);

        when(repository.findById(1L))
            .thenReturn(book);

        BookService service =
            new BookService(repository);

        Book result =
            service.getBook(1L);

        assertEquals("Clean Code", result.getTitle());

        verify(repository)
            .findById(1L);
    }
}
```

```
}  
}
```

The idea is:

```
BookService  
  ↓  
Mock BookRepository
```

instead of:

```
BookService  
  ↓  
Real Database
```

This makes service tests faster and isolated.

---

## 11. Logging

Your capstone should use logging instead of relying entirely on:

```
System.out.println();
```

For example:

```
private static final Logger logger =  
    LoggerFactory.getLogger(BookService.class);
```

Then:

```
logger.info("Adding book: {}", title);
```

And:

```
logger.error("Book not found: {}", id);
```

Don't log sensitive information.

Avoid things like:

```
password  
JWT token  
credit card information  
private credentials
```

---

## 12. Where Should You Log?

Useful places include:

### Service

```
Book created  
Book borrowed  
Book deleted
```

### Error handling

```
Book not found  
Database failure  
Unexpected exception
```

### Application startup

```
Application started
```

Don't log every insignificant line of code.

---

# 13. README

A professional capstone should have a [README.md](#).

It should tell another developer:

What is this project and how do I run it?

A good README contains:

```
Project name
Description
Features
Technologies
Requirements
Setup
Configuration
How to run
How to test
Sample requests
Sample responses
Project structure
```

Example:

```
# Library Management System

A Java backend application for managing books,
users, and borrowing operations.

## Features
```

- Add book
- Get all books
- Get book by ID
- Delete book
- Borrow book
- Return book

## ## Technologies

- Java
- Maven
- JUnit 5
- Mockito
- SLF4J

## ## Run

```
mvn clean package
```

```
java -jar target/library-management.jar
```

## ## Test

```
mvn test
```

---

# 14. Sample Input/Output

Include examples.

## Add Book

```
POST /books
```

Input:

```
{  
  "title": "Clean Code",
```

```
}
  "author": "Robert C. Martin"
}
```

Output:

```
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin"
}
```

## Get Book

```
GET /books/1
```

Output:

```
{
  "id": 1,
  "title": "Clean Code",
  "author": "Robert C. Martin"
}
```

## Book Not Found

```
GET /books/999
```

Response:

```
{
  "message": "Book not found"
}
```

---

## 15. Maven Build

Your project should build successfully with:

```
mvn clean package
```

You should understand:

```
mvn clean
```

removes previous build output.

```
mvn test
```

runs tests.

```
mvn package
```

compiles, tests, and packages the application.

---

## 16. Final Demo

Your final demonstration should follow this order:

1. Start application
- ↓
2. Show project structure
- ↓
3. Explain architecture
- ↓
4. Execute key feature
- ↓
5. Show successful response
- ↓

```
6. Show error case
  ↓
7. Run tests
  ↓
8. Explain one bug you fixed
  ↓
9. Explain design decisions
```

Don't simply say:

"Here is my project."

You should be able to **explain why you built it this way**.

---

## 17. Explain Your Design Decisions

An interviewer may ask:

Why did you create a service layer?

Answer:

I separated business logic from HTTP and data-access concerns. This makes the code easier to test, maintain, and modify.

---

### Why repository?

The repository isolates data-access logic from the service layer, so the service doesn't need to know how data is stored.

---

### Why DTO?

DTOs separate API input/output from internal domain objects and prevent exposing internal structures unnecessarily.

---

### Why custom exceptions?

They allow the application to represent specific failure cases clearly and handle them consistently.

---

## Why tests?

Tests verify business behavior and help prevent regressions when the code changes.

---

# 18. Explain One Bug You Fixed

This is a very common interview question.

Don't say:

"I never had bugs."

That's unrealistic.

Use:

```
Problem
↓
Investigation
↓
Root Cause
↓
Fix
↓
Prevention
```

For example:

## Problem

Book borrowing sometimes allowed an unavailable book to be borrowed.

## Root Cause

The service didn't check the available field before updating the book.

## Fix

Added a business rule:

```
if (!book.isAvailable()) {  
    throw new IllegalStateException("Book is unavailable");  
}
```

## Prevention

Added a unit test verifying that an unavailable book cannot be borrowed.

That's a **strong interview answer** because it demonstrates debugging + testing.

---

# 19. Interview Readiness – OOP

You should be able to answer:

## What are the four pillars of OOP?

```
Encapsulation  
Inheritance  
Polymorphism  
Abstraction
```

## Encapsulation?

Keeping data and behavior together while controlling access.

## Inheritance?

Creating a class based on another class.

## Polymorphism?

The same interface/reference can represent different implementations.

## Abstraction?

Hiding implementation details and exposing essential behavior.

---

## 20. Collections

Know the difference:

```
List  
Set  
Map
```

### ArrayList

Good general-purpose list.

```
List books = new ArrayList<>();
```

### HashSet

Stores unique values.

```
Set names = new HashSet<>();
```

### HashMap

Stores key-value pairs.

```
Map books = new HashMap<>();
```

Know basic complexity concepts too.

For example, HashMap lookup is  **$O(1)$  average case**, while searching an unsorted ArrayList is generally  **$O(n)$** .

---

## 21. Exceptions

You should know:

```
try
catch
finally
throw
throws
```

And:

```
Checked Exception
Unchecked Exception
```

Example:

```
try {
    service.getBook(10L);
} catch (BookNotFoundException e) {
    logger.error("Book not found", e);
}
```

---

## 22. Streams

You should understand operations such as:

```
books.stream()
    .filter(Book::isAvailable)
    .toList();
```

Know:

```
filter()
map()
sorted()
forEach()
collect()
toList()
```

And understand that streams are generally used for **processing collections**, not as a replacement for every loop.

---

## 23. Testing

Be ready for:

What is unit testing?

Testing a small unit of code in isolation.

What is Mockito?

A mocking framework used to create test doubles and isolate a class from its dependencies.

Why mock a repository?

So that a service can be tested without relying on a real database.

---

## 24. JDBC

You should be able to explain:

```
Connection
↓
PreparedStatement
↓
ResultSet
↓
Mapping
```

Know why:

```
PreparedStatement
```

is preferred over dynamically concatenating user input into SQL.

Main reason:

**SQL injection prevention and safer parameter handling.**

---

## 25. API Basics

You should know:

```
GET
POST
PUT
PATCH
DELETE
```

Common status codes:

```
200 OK
201 Created
204 No Content
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
405 Method Not Allowed
409 Conflict
500 Internal Server Error
```

Also understand:

```
JSON
DTO
Request
Response
Headers
Content-Type
Query parameters
Path variables
```

---

## 26. Final Interview Drill

You should be able to answer these without notes.

### Java

1. What is OOP?
2. Explain the four OOP pillars.
3. Difference between `==` and `.equals()`.
4. Difference between `ArrayList` and `LinkedList`.
5. Difference between `HashSet` and `HashMap`.
6. How does `HashMap` work conceptually?
7. Checked vs unchecked exceptions.
8. `throw` vs `throws`.
9. What is a Java Stream?
10. What is `Optional`?

### Testing

11. What is unit testing?
12. What is JUnit?
13. What is Mockito?
14. What is mocking?
15. Why mock a repository?
16. What is `verify()`?
17. What makes a good unit test?

### Database

18. What is JDBC?
19. What is `PreparedStatement`?
20. Why avoid SQL concatenation?
21. What is a transaction?
22. Explain ACID.
23. What is a primary key?
24. What is a foreign key?
25. Why use indexes?

### Backend/API

26. What is REST?
27. GET vs POST?
28. PUT vs PATCH?

- 29. What is JSON?
- 30. What is a DTO?
- 31. What is HTTP 404?
- 32. What is HTTP 405?
- 33. What is HTTP 500?
- 34. What is Content-Type?
- 35. Explain request lifecycle.

## Architecture

- 36. What is a monolith?
  - 37. What is layered architecture?
  - 38. What belongs in the service layer?
  - 39. What belongs in the repository?
  - 40. Why shouldn't controllers contain business logic?
  - 41. What is separation of concerns?
  - 42. What is dependency injection?
  - 43. Why build vertical slices?
- 

# 27. Final Capstone Checklist

Before submitting, verify:

- ☐ Project builds successfully
- ☐ Maven/Gradle configuration works
- ☐ Application starts successfully
- ☐ model package exists
- ☐ dto package exists where needed
- ☐ service package exists
- ☐ repository package exists
- ☐ exception handling exists
- ☐ app/main class exists
- ☐ Persistence works
- ☐ Main features work
- ☐ Invalid input is handled
- ☐ Meaningful errors are handled
- ☐ Service tests exist
- ☐ Important edge cases are tested
- ☐ Mockito tests exist if applicable
- ☐ Logging exists
- ☐ No sensitive data is logged
- ☐ README exists
- ☐ Setup instructions work

- ❑ Test instructions work
- ❑ Sample input/output exists
- ❑ Project structure is clean
- ❑ No unnecessary code
- ❑ No unnecessary dependencies

---

## 28. What You Should Be Able to Say After Completing This Roadmap

A strong final explanation would be:

**"I can build a Java backend application using a layered architecture. I understand OOP, collections, exceptions, streams, Maven/Gradle, SQL and JDBC, transactions, migrations, JUnit, Mockito, logging, DTOs, validation, error handling, and basic HTTP API design. I can separate controllers, services, and repositories, write meaningful tests, handle failures, and explain the design decisions behind my application."**

That's the actual goal of this roadmap.

---

## 29. Framework Readiness

After this foundation, moving to a framework becomes much easier.

You already understand:

```
HTTP
↓
Handler
↓
Service
↓
Repository
↓
Database
```

A framework can then simplify the infrastructure:

```
Spring Boot
  ↓
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

Instead of manually creating HTTP servers, routing requests, wiring objects, handling serialization, etc., the framework handles much of that infrastructure.

**That's why learning the fundamentals first is valuable.**

---

## 30. What Is Intentionally Outside This Roadmap?

You don't need to master everything immediately.

The roadmap intentionally leaves advanced topics such as:

```
Advanced JPA
GraphQL
Quarkus
Play Framework
Docker
Microservices
Kubernetes
Advanced distributed systems
```

for later.

The goal here is:

```
Core Java
  ↓
Backend Fundamentals
  ↓
```

```
graph TD; A[Testing] --> B[Database]; B --> C[HTTP/API]; C --> D[Architecture]; D --> E[Capstone]; E --> F[Framework Ready];
```

Testing  
↓  
Database  
↓  
HTTP/API  
↓  
Architecture  
↓  
Capstone  
↓  
Framework Ready

## Final checkpoint

If you can **build the capstone without copying a tutorial line-by-line**, explain why each layer exists, write tests for the important business rules, handle errors, and answer the interview questions above, then you're in a very good position to move into **Spring Boot and framework-based backend development**.